University of Applied Sciences and Arts
Northwestern Switzerland
School of Computer Science
Institute for Mobile and Distributed Systems (IMVS)

Master of Science in Engineering (MSE)

Project P8

Enhancing Web Accessibility Standards

Implementation of User-Centric Customization Interfaces and CSS-based Contrast Optimization

---

Author: **Florian Schnidrig**

Advisor: Prof. Dierk König

Profile: Computer Science

Date: Summer 2026 (TBD)

Contents

3. Introduction

The previous project, *Accessibility on the Web – Where we Stand* [1], examined the fundamental principles of digital inclusion within modern web applications. It explored how semantic HTML serves as the backbone for accessibility, provided an overview of both common and specialized native elements, and clarified the roles of ARIA and WCAG. Furthermore, it offered foundational guidance on accessibility debugging, testing, and the practical use of screen readers.

Building upon that foundation, this project addresses essential aspects of accessibility that, while perhaps less technical in a traditional coding sense, are no less vital for an inclusive digital world. While our previous focus remained largely on the structural and technical "under the hood" mechanics, we have yet to explore the visual design landscape. These design choices are far from purely aesthetic; they play a decisive role in determining whether a web application is a welcoming space or a digital dead end for many users.

4. Theory

4.1. CSS Media Queries

CSS Media Queries allow developer to apply different styles based on characteristics of a device or environment displaying a web page. This is often used to create a responsive web page that uses different stylings, based on screen width.

Listing 1 below describes the media query synthax according to W3C.

```
@media <media-query-list> {
  <rule-list>
}
```

Listing 1: Synthax of the media query where `<media-query-list>` contains `<media-type>`s, `<media-feature>`s as well as logical operators.

For web development the `screen` media-type is mainly of interest, but it is also possible to set it to `print` for targeting printers. The default value is `all`, representing both independent options and is suitable for all devices.

Media-feature is where it gets interesting as there are currently over 40 options available to listen and react to. For example, both `max-width` and `max-height`, as well as their opposites `min-width` and `min-height`, are widely used to create responsive web sites with different appearances optimized for different devices. Listing 2 showcases an example using `screen` and `max-width`. Some of these features consider aspects of accessibility and user preferences which make them essential to create an accessible web page considering individual needs of different users, but they are not as widely used yet as they should. The following media-feature values should be considered when building web applications that should include everyone. [2]

```
@media screen and (max-width: 768px) {
  body {
    background-color: lightskyblue;
  }
}
```

Listing 2: An example styling the background color for mobile devices with media-type `screen` and media-feature `max-width`

4.1.1. Reduced Motion

The `prefers-reduced-motion` media-feature indicated if a user has enabled a setting on their device to minimize the amount of non-essential motion shown to them. If the value is set to `reduced` animations and transitions should be reduced to the bare minimum.

A value set to `no-preference` indicated that the user has no preference on reduced motions, allowing all kinds of animations and movement. [3]

4.1.2. Contrast

With `prefers-contrast` the user can specify if he requests content to be presented with a lower or higher contrast. The value `no-preference` indicates that no setting regarding contrast preference was configured by the user.

If the value is set to `more` a higher level of contrast is requested by the user whilst a value of `less` indicated that the opposite, so a lower contrast, is preferred by the user.

Additionally, `custom` can be set as a value as well, meaning the user prefers a specific set of colors. The color palette is typically specified within the media feature `forced-colors` explained in the following section. [4]

4.1.3. Forced Colors

`forced-colors` detects, if a user agent has a forced colors mode enabled such as Windows High Contrast. This media-feature has either the value `active` if such a mode is enabled or `none` if not.

When activated, this overwrites and restricts colors defined in the style sheet. Color values are not affected by styles defined in CSS but instead forced by the browser at paint time. The colors enforced depend on the context of an element. Additionally, backplates are drawn behind text to ensure legibility to preserve contrast for text placed on top of images.

Developers are not encouraged to use this media query to create a separate design for users with this feature enabled but to make small tweaks to improve usability if a certain part of a page would not work well in the forced colors context such as replacing box-shadow stylings with a border to not lose the contrast of a button that is only elevated from the background with a box-shadow. [5]

4.1.4. Reduced Transparency

When a user has enabled a setting to reduce the transparent or translucent layer effects the `prefers-reduced-transparency` media-feature will be set to `reduced` while a value of `no-preference` indicates default behavior is expected by the user. This can help improve contrast and readability for some users.

This feature is restricted available and not yet fully supported by Firefox and Safari. [6]

4.1.5. Color Scheme

A feature more broadly used compared to the others introduced in this chapter is the `prefers-color-scheme`. It indicates if the user requests a light or dark color theme

set through the operating system or a user agent setting. Possible values are `light` for a light color theme and `dark` for a dark color theme. [7]

4.1.6. Inverted Colors

Using inverted colors can have unpleasant side effects such as shadows turning into highlights, reducing the readability of the content. Developers can react to this preference and ensure the pages integrity with this media-feature. This feature is only supported by Safari so far. [8]

4.1.7. Accessing Media Features through JavaScript

The `window` interface provides the `matchMedia()` method, returning a `MediaQueryList` object to check if the `document` matches a media query string as seen below in Listing 3 for the `prefers-color-scheme` query.

```
const isDarkTheme = window.matchMedia("(prefers-color-scheme:
dark)").matches;
```

Listing 3: Checking if the user's system settings prefere a dark color scheme

This enables developers to not only react to preferences within CSS but also consider the users wishes within business logic and web page specific features. [9]

4.1.8. Changes in the World of Media Queries

Continuous development and improvement take place in the world of CSS media queries but there is currently one big visual accessibility concern that is not considered yet. Different types of colorblindness and other visual impairments in regards of color perception fall short. There is an open issue in the W3Cs csswg-drafts repository proposing an additional `media-feature` called `color-vision-adjustment` that intends to cover that area to enhance web accessibility for users with color vision deficiencies.

This proposal intends to cover various specific types of colorblindness such as `protanopia` (red deficiency), `deuteranopia` (red-green deficiency), `tritanopia` (blue-yellow deficiency) or `achromatopsia` (near total color blindness - grayscale) that were covered in the previous P7 project with bookmarklets simulating these visual impairments.

Having such a `media-feature` would increase the awareness as well as the possibilities to directly react to the needs of users affected which such impairments. [10]

4.1.9. Deprecated Features

There were media types considering accessibility needs like `embossed`, `aural` and `braille`, but they got deprecated, assuming distinctions between device types will

become more blurred in the future and due to media-type referring to device categories rather than the media they support. [11], [12]

4.1.10. Security Concerns

Data accessible with media queries can be abused to construct a fingerprint, helping to identify and track a device. To prevent this, browsers may fudge returned values in some manners. Some Browsers also provide the possibility to prevent this abuse in the browser settings, such as “Resist Fingerprinting” in Firefox resulting in many media queries only reporting default values rather than device specific settings. [12]

4.2. CSS Custom Properties

CSS custom properties, also referred to as CSS variables, are entities representing values. These properties can be used throughout the document, evaluating into the same values, depending on the scope it is defined, everywhere they are used. This helps developers to keep an overview and reduces complexity. Such a property is typically set by the custom property syntax, being two dashes `--`, followed by a name, joined with single dashes. To access a custom property the CSS `var()` function must be used. Listing 4 shows an example of custom properties being used to define a `--primary-color` that then can be used on the whole website and is easy exchangeable in one central part to implement different color palates such as a light and dark theme, based on the users preference. [13]

```css
:root {
  --primary-color: #204CCF;
  --primary-color-contrast: #FFFFFF;
}

button.primary {
  background-color: var(--primary-color);
  color: var(--primary-color-contrast);
}

h1, h2, h3 {
  color: var(--primary-color);
}
```

Listing 4: Defining a primary color custom propperty that can be used throughout the website

It is possible to be more precise when defining a custom property by using the `@property` at-rule, allowing to define the value type with `syntax`, if the property inherits by default with `inherits` and an initial value with `initial-value`. There is a wide range of values possible for `syntax` such as `<color>`, `<number>` and `<url>` to name a few. Listing 5 shows how the `---primary-color` example from before is achieved with this at-rule. [14], [15]

```css
@property --primary-color {
  syntax: "<color>";
  inherits: false;
  initial-value: #204CCF;
}
```

Listing 5: Defining a primary color custom propperty using the `@property` rule to be more precise

As displayed in Listing 6 below, both variations are also definable with JavaScript using `registerPropperty()` or `setProperty()`. [16], [17]

```
window.CSS.registerProperty({
  name: '--primary-color',
  syntax: '<color>',
  inherits: false,
  initialValue: '#204CCF ',
});

const root = document.documentElement;
root.style.setProperty('--primary-color-contrast', '#FFFFFF');
```

Listing 6: Using both `registerProperty()` and `setProperty()` in JavaScript to create CSS custom properties

4.2.1. Global State Management

[TODO Add `@container` and `[style*="--custom-property: value"]` to dive into State management.]

4.3. CSS Functions

4.4. Contrast perception

[TODO] Some introducing summary on contrast.

4.4.1. Relative Luminance

The current WCAG 2.x standard bases contrast requirements on relative luminance between text and its background, independent of hue. This assumes that for individuals with color vision deficiencies, hue and saturation have minimal or no effect regarding legibility as assessed by reading performance. Since the inability to distinguish specific colors does not typically impair light-dark perception, color itself is not considered a primary factor in these calculations. Far more important is that smaller and thinner fonts may be rendered by user agents with a much fainter appearance than the color defined in the CSS, leading to a perceived contrast that is notably lower than the theoretical ratio.

The WCAG guideline requires at least a contrast ratio of 3:1 to satisfy the corelating requirement for level AA. This is based in the recommendation from ISO-9241-3. To achieve level AAA certification the minimum ratio is set to 7:1. This ratio was chosen because it compensated for the loss in contrast sensitivity experienced by users with approximately 20/80 vision (This means that someone needs to be 20 feet away to see what a person with normal vision can see from 80 feet away). People with more than this degree of vision loss usually use assistive technologies.

Color deficiencies are so diverse that it is impossible to generalize effective color pairs for contrast based on quantitative data. This is why contrast independent of color perception is so important.

A notable exception is protanopia where red color tones are perceived as dark grey, resulting in a bad contrast on darker colors and black. This is why it is recommended to generally not use red on black. [18]

The formula depicted in Listing 7 is used in the WCAG 2.x specification to calculate the relative luminance used for further calculations such as color contrast. [19]

$$L = 0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B$$

$$\text{Where } C = \begin{cases} \frac{C_{\text{sRGB}}}{12.92} & \text{if } C_{\text{sRGB}} \leq 0.03928 \\ \left(\frac{C_{\text{sRGB}}+0.055}{1.055}\right)^{2.4} & \text{otherwise} \end{cases}$$

Example: Dodger Blue `rgb(30, 144, 255)`

1. Normalize (/255)

$R_{\text{sRGB}} = \frac{30}{255} = 0.1176$
$G_{\text{sRGB}} = \frac{144}{255} = 0.5647$
$B_{\text{sRGB}} = \frac{255}{255} = 1.0000$

2. Linearize

$R = 0.0123$
$G = 0.2747$
$B = 1.0000$

3. Calculate Luminance (L)

$$L = (0.2126 \cdot 0.0123) + (0.7152 \cdot 0.2747) + (0.0722 \cdot 1.0000) = \mathbf{0.2712}$$

Calculate Contrast Ratio

$$\text{Ratio} = \frac{\text{L1} + 0.05}{\text{L2} + 0.05}$$

Listing 7: Step-by-step calculation of relative luminance based on the W3C sRGB formula. [19]

4.4.2. Perceptual Color Models

WCAG 3.0, which is currently still in development, replaces the WCAG 2.x contrast methods with Advanced Perceptual Contrast Algorithm (APCA). [20]

4.5. Ishihara

5. Implementation

5.1. Preferences Weidget

5.2. Contrast Color Functions

5.3. Interactive Ishihara Plate

6. Discussion

7. Conclusions

Bibliography

[1] Florian Schnidrig, "Accessibility on the Web – Where We Stand." Accessed: Apr. 13, 2026. [Online]. Available: https://accessible-web-initiative.gitbook.io/accessibility-on-the-web-where-we-stand

[2] Mozilla Corporation, "Using media queries." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries/Using

[3] Mozilla Corporation, "prefers-reduced-motion." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion

[4] Mozilla Corporation, "prefers-contrast." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-contrast

[5] Mozilla Corporation, "forced-colors." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/forced-colors

[6] Mozilla Corporation, "prefers-reduced-transparency CSS media feature." Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-transparency

[7] Mozilla Corporation, "prefers-color-scheme." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

[8] Mozilla Corporation, "inverted-colors." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

[9] Mozilla Corporation, "Window: matchMedia() method." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/matchMedia

[10] Naftali Peles, "[mediaqueries] CSS Media Feature for Color Vision Adjustments." Accessed: Mar. 16, 2026. [Online]. Available: https://github.com/w3c/csswg-drafts/issues/10335

[11] World Wide Web Consortium, "Media Queries – disposition of comments." Accessed: Apr. 16, 2026. [Online]. Available: https://www.w3.org/Style/css3-updates/css3-mediaqueries-comments

[12] Mozilla Corporation, "@media." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media#media_types

[13] Mozilla Corporation, “Using CSS custom properties (variables).” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascading_variables/Using_custom_properties

[14] Mozilla Corporation, “@property CSS at-rule.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@property

[15] Mozilla Corporation, “syntax CSS at-rule descriptor.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@property/syntax

[16] Mozilla Corporation, “CSS: registerProperty() static method.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/CSS/registerProperty_static

[17] Mozilla Corporation, “CSSStyleDeclaration: setProperty() method.” Accessed: Apr. 23, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/CSSStyleDeclaration/setProperty

[18] World Wide Web Consortium, “Contrast (Minimum) (Level AA).” Accessed: Apr. 20, 2026. [Online]. Available: https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html

[19] World Wide Web Consortium, “Relative luminance.” Accessed: Apr. 20, 2026. [Online]. Available: https://www.w3.org/WAI/GL/wiki/Relative_luminance

[20] Andrew Somers, “Why APCA as a New Contrast Method?.” [Online]. Available: https://git.apcacontrast.com/documentation/WhyAPCA.html

List of Figures

List of Aids

Artificial Intelligence and Language Models

Tool: Gemini (Google), Version 3 Flash.

Type of Usage: Used for linguistic refinement of drafts and formating mathematical formulas (Relative Luminance).

Extent: Specific paragraphs regarding WCAG luminance calculations were refined using the tool. All technical facts and code were manually verified for accuracy.